Experiment No. 8

Experiment Name - Customer Churn Prediction using Decision Tree Classification

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 14

CU ID : CU24250231

1. Select a suitable dataset such as the Telco Customer Churn Dataset, Bank Customer Churn Dataset, or IBM HR Employee Attrition Dataset.

```
# Selecting a Real-World Dataset
from google.colab import files
uploaded = files.upload()

import os
print(os.listdir())

dataset_name = "Telco Customer Churn Dataset"

print("Selected Dataset:", dataset_name)
print("Dataset selected successfully!")
```

```
Choose Files Telco_Cuso…r_Churn.csv
Telco_Cusomer_Churn.csv(text/csv) - 977501 bytes, last modified: 9/20/2026 - 100% done
Saving Telco_Cusomer_Churn.csv to Telco_Cusomer_Churn.csv
['.config', 'Telco_Cusomer_Churn.csv', 'sample_data']
Selected Dataset: Telco Customer Churn Dataset
Dataset selected successfully!
```

2. Import the required Python libraries and load the dataset into a Pandas DataFrame.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
from sklearn.tree import plot_tree

# Load the dataset
df = pd.read_csv("Telco_Cusomer_Churn.csv")

# Display first 5 rows
print("First 5 rows of the dataset:")
display(df.head())

# Display dataset shape
print("Dataset Shape:", df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())
```

```
First 5 rows of the dataset:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecur
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
Dataset Shape: (7043, 21)

Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetServi
```

3. Perform data preprocessing by handling missing values, encoding categorical variables, and selecting relevant features for prediction.

```
# Check Dataset Information
print("Dataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
```

```
Churn              0
dtype: int64
```

```
# Convert TotalCharges to Numeric
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

print("Data type of TotalCharges after conversion:")
print(df["TotalCharges"].dtype)

print("\nMissing values in TotalCharges:")
print(df["TotalCharges"].isnull().sum())
```

```
Data type of TotalCharges after conversion:
float64

Missing values in TotalCharges:
11
```

```
# Handle Missing Values
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())

print("Missing values after handling:")
print(df.isnull().sum().sum())
```

```
Missing values after handling:
0
```

```
# Remove Customer ID
df = df.drop("customerID", axis=1)

print("Customer ID removed successfully.")
print("New dataset shape:", df.shape)
```

```
Customer ID removed successfully.
New dataset shape: (7043, 20)
```

```
# Encode Categorical Variables
df = pd.get_dummies(df, drop_first=True)

print("Categorical variables encoded successfully.")

print("\nNew dataset shape:", df.shape)

print("\nFirst 5 rows after encoding:")
display(df.head())
```

```
Categorical variables encoded successfully.

New dataset shape: (7043, 31)

First 5 rows after encoding:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	gender_Male	Partner_Yes	Dependents_Yes	PhoneService_Yes	Multiple phone
0	0	1	29.85	29.85	False	True	False	False	
1	0	34	56.95	1889.50	True	False	False	True	
2	0	2	53.85	108.15	True	False	False	True	
3	0	45	42.30	1840.75	True	False	False	False	
4	0	2	70.70	151.65	False	False	False	True	

5 rows × 31 columns

```
# Separate Features and Target
X = df.drop("Churn_Yes", axis=1)
y = df["Churn_Yes"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

print("\nTarget values:")
print(y.value_counts())
```

```
Features shape: (7043, 30)
Target shape: (7043,)

Target values:
Churn_Yes
False    5174
```

```
True     1869
Name: count, dtype: int64
```

4. Split the dataset into training and testing sets using an appropriate train-test ratio (e.g., 80:20).

```
# Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

print("\nTraining target distribution:")
print(y_train.value_counts())

print("\nTesting target distribution:")
print(y_test.value_counts())
```

```
Training set size: (5634, 30)
Testing set size: (1409, 30)

Training target distribution:
Churn_Yes
False    4139
True     1495
Name: count, dtype: int64

Testing target distribution:
Churn_Yes
False    1035
True      374
Name: count, dtype: int64
```

5. Train a Decision Tree Classifier using the training dataset and configure suitable parameters such as maximum tree depth and splitting criteria (Gini Index or Entropy).

```
# Train Decision Tree Classifier
decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

# Train the model
decision_tree.fit(X_train, y_train)

print("Decision Tree Classifier trained successfully!")
```

```
Decision Tree Classifier trained successfully!
```

6. Use the trained model to predict customer churn for the testing dataset.

```
# Predict Customer Churn
y_pred = decision_tree.predict(X_test)

print("Predictions generated successfully!")

print("\nFirst 20 Actual Values:")
print(y_test.values[:20])

print("\nFirst 20 Predicted Values:")
print(y_pred[:20])
```

```
Predictions generated successfully!

First 20 Actual Values:
[False False False False False False False False False  True False False
 False  True False False False False False False]

First 20 Predicted Values:
[False  True False False False False False False False False False False
 False  True False False False  True False False]
```

7. Evaluate the model using a Confusion Matrix, Accuracy, Precision, Recall, and F1- Score.

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

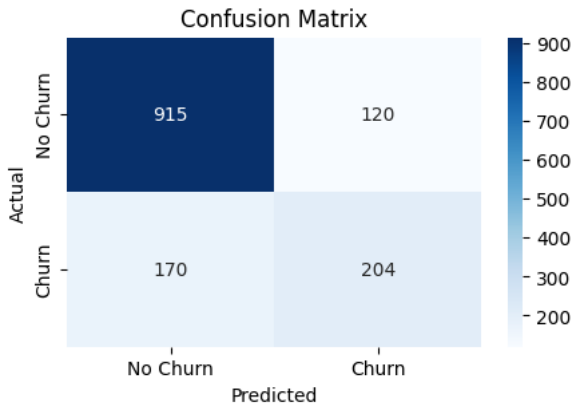
print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[915 120]
 [170 204]]
```

```
# Visualize Confusion Matrix
plt.figure(figsize=(5, 3))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

```
Accuracy: 0.794180269694819
Accuracy Percentage: 79.4180269694819 %
```

```
# Precision
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
```

```
Precision: 0.6296296296296297
```

```
# Recall
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
```

```
Recall: 0.5454545454545454
```

```
# F1-Score
f1 = f1_score(y_test, y_pred)
print("F1-Score:", f1)
```

```
F1-Score: 0.5845272206303725
```

```
# Display All Evaluation Metrics
print("Decision Tree Classification Performance\n")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
```

```
Decision Tree Classification Performance

Accuracy : 0.794180269694819
Precision: 0.6296296296296297
Recall   : 0.5454545454545454
F1-Score : 0.5845272206303725
```

```
# Classification Report
print("Classification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["No Churn", "Churn"]
))
```

```
Classification Report:
              precision    recall  f1-score   support

    No Churn       0.84      0.88      0.86      1035
       Churn       0.63      0.55      0.58       374

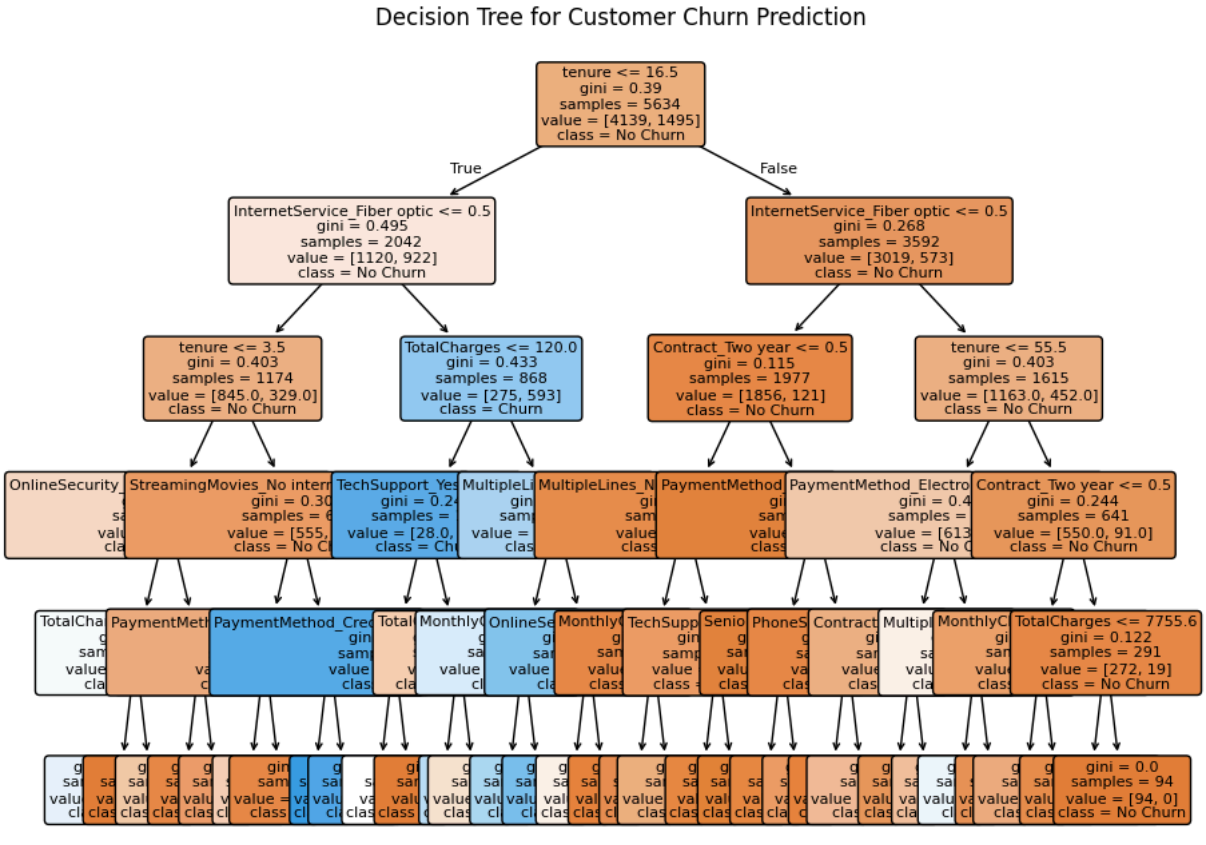
    accuracy                           0.79      1409
   macro avg       0.74      0.71      0.72      1409
weighted avg       0.79      0.79      0.79      1409
```

8. Visualize the Decision Tree to understand the decision-making process and identify the most influential features.

```
# Visualize Decision Tree
plt.figure(figsize=(10, 8))

plot_tree(
    decision_tree,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Customer Churn Prediction")
plt.show()
```



9. Analyze the classification results and determine the key factors contributing to customer churn.

```
# Feature Importance
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": decision_tree.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("Feature Importance:")
display(feature_importance)
```

```
Feature Importance:
```

	Feature	Importance
1	tenure	0.421435
10	InternetService_Fiber optic	0.357542
3	TotalCharges	0.047129
28	PaymentMethod_Electronic check	0.036761
9	MultipleLines_Yes	0.023630
12	OnlineSecurity_No internet service	0.020669
25	Contract_Two year	0.019599
22	StreamingMovies_No internet service	0.017379
2	MonthlyCharges	0.016847
19	TechSupport_Yes	0.012988
24	Contract_One year	0.008553
8	MultipleLines_No phone service	0.007049
13	OnlineSecurity_Yes	0.006628
27	PaymentMethod_Credit card (automatic)	0.002533
7	PhoneService_Yes	0.000975
0	SeniorCitizen	0.000283
4	gender_Male	0.000000
5	Partner_Yes	0.000000
11	InternetService_No	0.000000
6	Dependents_Yes	0.000000
16	DeviceProtection_No internet service	0.000000
14	OnlineBackup_No internet service	0.000000
21	StreamingTV_Yes	0.000000
20	StreamingTV_No internet service	0.000000
18	TechSupport_No internet service	0.000000
17	DeviceProtection_Yes	0.000000
15	OnlineBackup_Yes	0.000000
23	StreamingMovies_Yes	0.000000
26	PaperlessBilling_Yes	0.000000
29	PaymentMethod_Mailed check	0.000000

Next steps: Generate code with `feature_importance` New interactive sheet

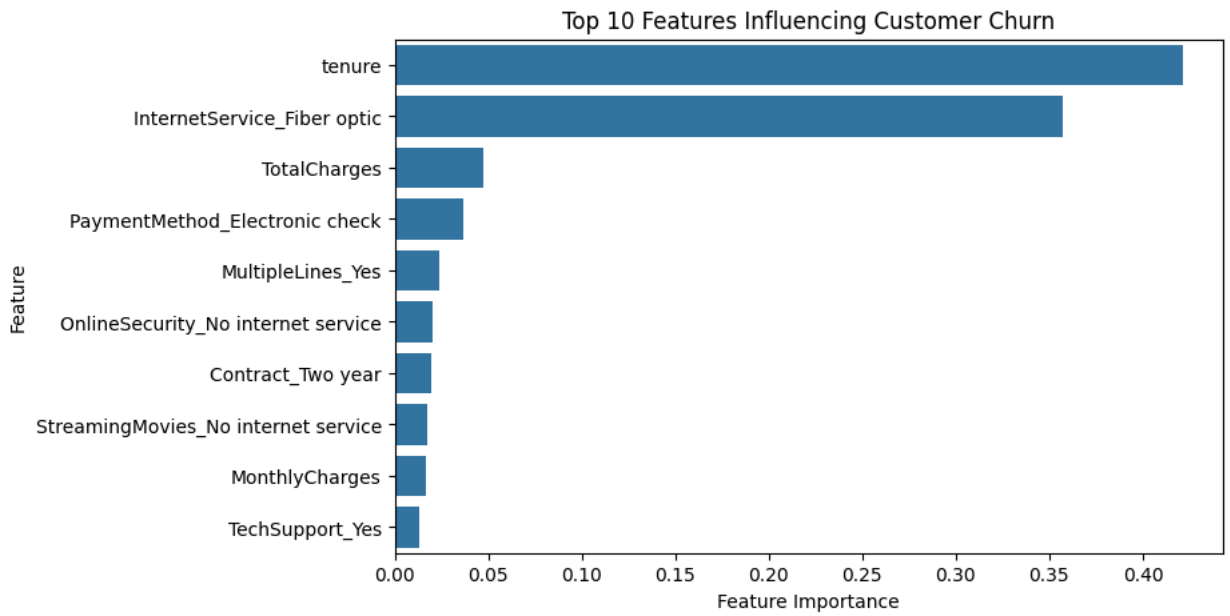
```
# Visualize Feature Importance
top_features = feature_importance.head(10)

plt.figure(figsize=(8, 5))

sns.barplot(
    data=top_features,
    x="Importance",
    y="Feature"
)

plt.title("Top 10 Features Influencing Customer Churn")
```

```
plt.xlabel("Feature Importance")
plt.ylabel("Feature")
plt.show()
```




```
# Final Model Analysis
print("CUSTOMER CHURN PREDICTION - DECISION TREE\n")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nTop 10 Important Features:")
display(feature_importance.head(10))
```

```
CUSTOMER CHURN PREDICTION - DECISION TREE

Accuracy  : 0.7942
Precision : 0.6296
Recall    : 0.5455
F1-Score  : 0.5845

Top 10 Important Features:
```

	Feature	Importance
1	tenure	0.421435
10	InternetService_Fiber optic	0.357542
3	TotalCharges	0.047129
28	PaymentMethod_Electronic check	0.036761
9	MultipleLines_Yes	0.023630
12	OnlineSecurity_No internet service	0.020669
25	Contract_Two year	0.019599
22	StreamingMovies_No internet service	0.017379
2	MonthlyCharges	0.016847
19	TechSupport_Yes	0.012988

10. Summarize the findings and recommend business strategies for improving customer retention based on the model's predictions.

The Decision Tree Classification model can be used to identify customers who are more likely to churn based on factors such as contract type, tenure, monthly charges, payment method, and subscribed services. The model helps in understanding the important factors that influence customer churn and provides useful information for customer retention.

Based on the model's predictions, the company can take the following strategies to improve customer retention:

1. Identify customers with a high probability of churn and provide them with targeted retention offers.
2. Provide discounts, loyalty benefits, and personalized plans to customers who are likely to leave.
3. Encourage customers to choose long-term contracts by offering attractive benefits and reduced pricing.

4. Improve customer support and technical assistance, especially for customers experiencing service-related problems.
5. Provide additional services such as online security, backup, and technical support according to customer requirements.
6. Monitor customers with low tenure and provide special onboarding support to build long-term relationships.
7. Analyze customer feedback and complaints regularly to identify and resolve common reasons for dissatisfaction.

In conclusion, Decision Tree Classification provides an interpretable approach to predicting customer churn and identifying important factors affecting customer retention. By using these predictions, businesses can take timely and personalized actions to improve customer satisfaction, reduce customer churn, and maintain long-term customer relationships.

QUESTIONS AND ANSWERS

1. What is classification, and how does it differ from regression?

- Classification is a supervised machine learning technique used to predict a categorical output or class label. For example, in customer churn prediction, the output can be "Churn" or "No Churn".
- Regression is used to predict a continuous numerical value. For example, predicting house price or salary is a regression problem.
- Thus, classification predicts categories, whereas regression predicts numerical values.

2. Explain the working principle of the Decision Tree algorithm.

- A Decision Tree is a supervised machine learning algorithm that makes predictions by dividing the dataset into smaller groups using a series of decision rules.
- The algorithm starts with a root node and selects the best feature for splitting the data using criteria such as Gini Index or Entropy. The process continues through internal nodes until the data reaches a leaf node, which represents the final predicted class.
- For example, in customer churn prediction, the tree may first check the contract type, followed by tenure or monthly charges, and finally predict whether the customer is likely to churn.

3. Differentiate between Gini Index and Entropy as splitting criteria.

- Gini Index and Entropy are measures used by Decision Trees to determine the best feature for splitting the data.
- Gini Index measures the probability of incorrectly classifying a randomly selected sample if it is labeled according to the distribution of classes in the node. A lower Gini Index indicates a purer node.
- Entropy measures the amount of uncertainty or impurity in a dataset. A lower entropy indicates a purer node.
- The main difference is that Gini Index is generally simpler and computationally faster, while Entropy is based on information gain and involves logarithmic calculations. Both criteria are used to select effective splits in a Decision Tree.

4. What is a Confusion Matrix? Explain its components.

A Confusion Matrix is a table used to evaluate the performance of a classification model by comparing actual classes with predicted classes.

It consists of four components:

- True Positive (TP): The model correctly predicts the positive class.
- True Negative (TN): The model correctly predicts the negative class.
- False Positive (FP): The model incorrectly predicts a positive class when the actual class is negative.
- False Negative (FN): The model incorrectly predicts a negative class when the actual class is positive.

The Confusion Matrix helps in understanding the types of errors made by a classification model.

5. Define Accuracy, Precision, Recall, and F1-Score. Why are these metrics important?

- Accuracy is the proportion of correctly classified observations out of the total observations.
- Precision is the proportion of correctly predicted positive observations among all observations predicted as positive.
- Recall is the proportion of correctly predicted positive observations among all actual positive observations.
- F1-Score is the harmonic mean of Precision and Recall and provides a balanced measure of both.

These metrics are important because accuracy alone may not provide a complete picture of classification performance, especially when the classes are imbalanced. Precision, Recall, and F1-Score provide additional information about the model's classification errors and ability to identify positive cases.

6. What is overfitting in a Decision Tree? How can it be reduced?

- Overfitting occurs when a Decision Tree becomes too complex and learns the training data too closely, including its noise and minor patterns. As a result, the model may perform well on training data but poorly on unseen testing data.
- Overfitting can be reduced by limiting the maximum depth of the tree, setting a minimum number of samples required for splitting, setting a minimum number of samples required at leaf nodes, and pruning unnecessary branches.

- Using suitable train-test splitting and cross-validation can also help in controlling overfitting.

7. Why is customer churn prediction important for businesses?

- Customer churn prediction is important because losing existing customers can negatively affect business revenue and growth. A churn prediction model helps businesses identify customers who are likely to discontinue their services.
- By identifying such customers early, businesses can take appropriate actions such as providing personalized offers, improving customer support, and addressing customer concerns.
- Therefore, churn prediction helps organizations improve customer retention, customer satisfaction, and long-term business performance.

8. What are the advantages and limitations of the Decision Tree algorithm?

Advantages:

- Decision Trees are easy to understand and interpret.
- They can handle both numerical and categorical features.
- They require relatively little data preprocessing.
- The decision-making process can be visualized easily.
- They can identify important features that influence predictions.

Limitations:

- Decision Trees can easily overfit the training data.
- Small changes in the data can sometimes produce a different tree.
- A very large tree can become difficult to interpret.
- They may not always provide the best predictive performance compared with some other machine learning algorithms.
- Complex trees may require pruning or parameter tuning.

9. Mention three real-world applications of Decision Tree Classification other than customer churn prediction.

Three real-world applications of Decision Tree Classification are:

- Medical Diagnosis — Predicting whether a patient has a particular disease based on symptoms and medical information.
- Credit Risk Assessment — Classifying loan applicants as low-risk or high-risk based on financial and personal information.
- Spam Email Detection — Classifying emails as spam or legitimate based on characteristics of the messages.

10. How can the insights obtained from a churn prediction model help organizations improve customer retention and business profitability?

- A churn prediction model helps organizations identify customers who are more likely to leave and understand the factors associated with customer churn.
- Organizations can use these insights to provide personalized offers, discounts, loyalty programs, better customer support, and suitable service plans to at-risk customers. They can also identify common problems affecting customer satisfaction and take corrective actions.
- By retaining existing customers, reducing unnecessary customer loss, and focusing retention efforts on customers who need attention, organizations can improve customer satisfaction, maintain recurring revenue, and increase overall business profitability.